T3000 API & Protocol

Schedules · built Nov 13, 2026

Schedules

Time-based control schedules for automated equipment operation.

Overview

Schedules automate equipment operation based on time of day and day of week. Common uses include occupancy schedules, setpoint changes, and equipment start/stop times.

Schedule Types

- **Weekly Schedules**: Different times for each day of week
- **Exception Schedules**: Override for holidays/special events
- **Annual Schedules**: Yearly calendar events

Creating Schedules

1. Navigate to **Schedules** page
2. Click **Add Schedule**
3. Set times for each day
4. Assign to outputs/variables
5. Enable schedule

Example Schedule

Monday-Friday:

- 6:00 AM: Occupied mode, 72F
- 6:00 PM: Unoccupied mode, 65F

Saturday-Sunday:

- All day: Unoccupied mode, 60F

Next Steps

- Holidays - Holiday/exception schedules

- Programs - Schedule programs

Schedules

Schedule API

Schedule Data Model

TYPESCRIPT

```typescript
interface Schedule {
  id: number;
  name: string;
  enabled: boolean;
  weeklySchedule: DaySchedule[];
  exceptions: ExceptionSchedule[];
}

interface DaySchedule {
  dayOfWeek: 0 | 1 | 2 | 3 | 4 | 5 | 6;  // 0 = Sunday
  events: ScheduleEvent[];
}

interface ScheduleEvent {
  time: string;  // "HH:MM"
  value: number;
}

// Get all schedules
const schedules = await fetch(
  `http://localhost:9103/api/schedules`
).then(r => r.json());

// Create schedule
await fetch(`http://localhost:9103/api/schedules`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    name: 'Office Occupancy',
    enabled: true,
    weeklySchedule: [
      {
        dayOfWeek: 1,  // Monday
        events: [
```

```
          { time: '06:00', value: 72 },
          { time: '18:00', value: 65 }
        ]
      }
    ]
  })
});
```

Schedule Execution Engine

TYPESCRIPT

```typescript
class ScheduleExecutor {
  private schedules = new Map<number, Schedule>();
  private timers = new Map<number, NodeJS.Timeout>();

  start(schedule: Schedule) {
    // Calculate next event time
    const nextEvent = this.findNextEvent(schedule);
    if (!nextEvent) return;

    const delay = nextEvent.time - Date.now();

    const timer = setTimeout(() => {
      this.executeEvent(schedule, nextEvent);
      this.start(schedule);  // Schedule next event
    }, delay);

    this.timers.set(schedule.id, timer);
  }

  private findNextEvent(schedule: Schedule) {
    const now = new Date();
    const currentDay = now.getDay();
    const currentTime = `${now.getHours().toString().padStart(2, '0')}:${now.getMinutes(

    // Check today's remaining events
    const todaySchedule = schedule.weeklySchedule.find(
      s => s.dayOfWeek === currentDay
    );

    if (todaySchedule) {
      const remainingEvents = todaySchedule.events.filter(
        e => e.time > currentTime
      );

      if (remainingEvents.length > 0) {
        return {
          time: this.parseTime(remainingEvents[0].time),
          value: remainingEvents[0].value
```

```
        };
      }
    }

    // Find next day's first event
    for (let i = 1; i <= 7; i++) {
      const nextDay = (currentDay + i) % 7;
      const daySchedule = schedule.weeklySchedule.find(
        s => s.dayOfWeek === nextDay
      );

      if (daySchedule && daySchedule.events.length > 0) {
        return {
          time: this.parseTime(daySchedule.events[0].time, i),
          value: daySchedule.events[0].value
        };
      }
    }

    return null;
  }
}
```

BACnet Schedule Object

TYPESCRIPT

```typescript
// Read BACnet schedule
const schedule = await client.readProperty({
  deviceInstance: 389001,
  objectType: 'schedule',
  objectInstance: 1,
  property: 'weekly-schedule'
});

// Write schedule
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'schedule',
  objectInstance: 1,
  property: 'weekly-schedule',
  value: {
    monday: [
      { time: { hour: 6, minute: 0 }, value: 72 },
      { time: { hour: 18, minute: 0 }, value: 65 }
    ]
  }
});
```

Next Steps

- REST API
- BACnet Objects
- Programs